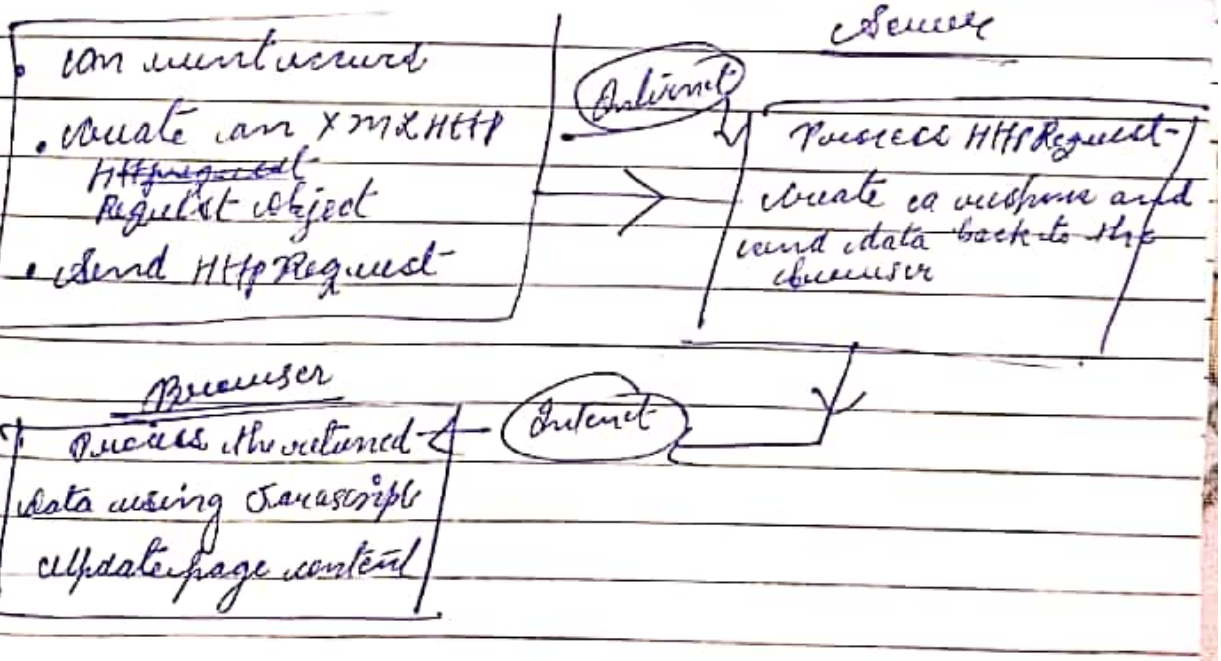
AJAX

Asynchronous JavaScript and XML
AJAX just uses of combination of
a browser built in XMLHttpRequest object
(to request data from a web server)
JavaScript and HTML DOM (to display or use the
data)

- Ajax allows web pages to update asynchronously by exchanging data with a web server behind the scene.
- Update part of a webpage without reloading the whole page.



Module — Contains open-source packages to include 3rd party code in our code (e.g. React, jQuery, Leaflet etc)

npm node JS

Module → Bundling → Transpiling polyfilling

Bundling: join all modules into one

Transpiling polyfilling → Javascript bundle

we end of up in a callback hell
(Pyramid of doom)

* Inversion of control =

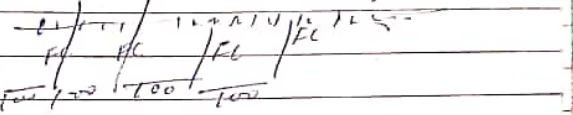
for proceed payment callback function to proceed we need card function to run, we are dependent on card function for proceed to payment to be executed. what happens if the function doesn't run. this called inversion of control. giving control to some other function.

Example 3

Button → click
(clickhandler)

clicked multiple times
(shooting games)
"" — "" "" "" — "" diff b/w the click & 500ms
(debouncing)

Throttling (after interval of 100ms)



Example 4 → Scrolling event.

which is better depends on situation or scenario.

Debouncing

Create a html page
<body>

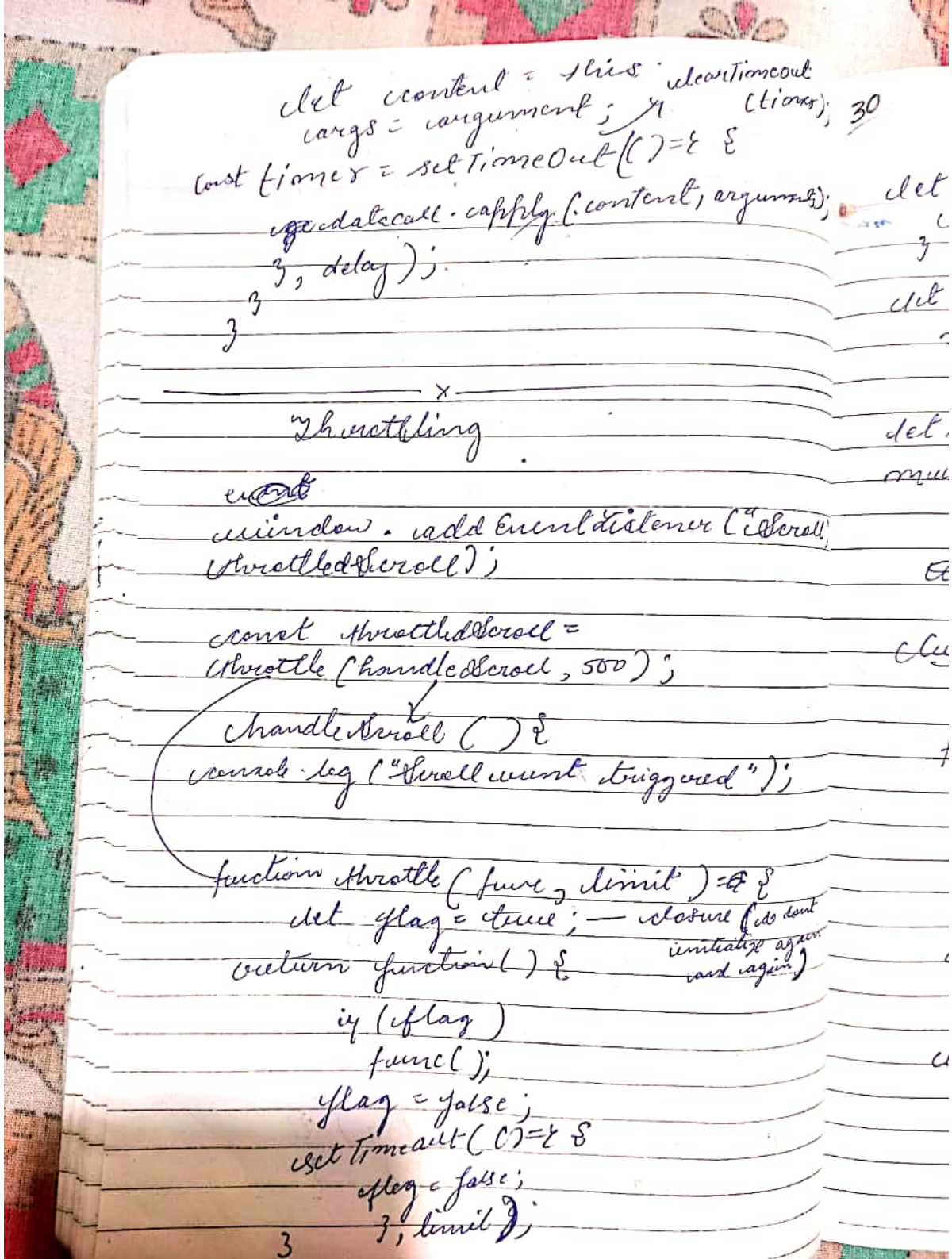
<input type="text" name="text" onkeyup="datacall()" />

</body>
let c=0
datacall(e) {
console.log("API call", c++);
}

const result = debounce(datacall, 300)

function debounce(fn, delay) {
return function() {

Scanned with CamScanner



```
let context = this;
args = arguments; // clearTimeout(timer); 30
const timer = setTimeout(() => {
  datacall.apply(context, args);
}, delay);
};
```

Throttling

```
window.addEventListener("scroll",
throttledScroll);

const throttledScroll =
throttle(handleScroll, 500);

handleScroll() {
console.log("Scroll event triggered");

function throttle(func, limit) {
  let flag = true; — closure (do dont initialize again and again)
  return function() {
    if (flag)
      func();
    flag = false;
    setTimeout(() => {
      flag = false;
    }, limit);
};
```

Polyfill for bind

```
let name = {
  firstname: "Akshay",
  lastname: "Saini"
};
let printname = function () {
  console.log(this.firstname + " " + this.lastname);
};

let print = printname.bind(name);
```

callable to call functions

```
let print2 = printname.mybind(name)

Function.prototype.mybind =
  function (...args) {
    let obj = this;
    return function () {
      obj.call(args[0]);
    };
  };
```

(holding argument name)

this print to method print name

advance polyfill for bind

```
function.prototype.mybind = function
                              (...args) {
  let obj = this,
  params = args.slice(1);
```

args.slice(1);

30

Currying in Javascript

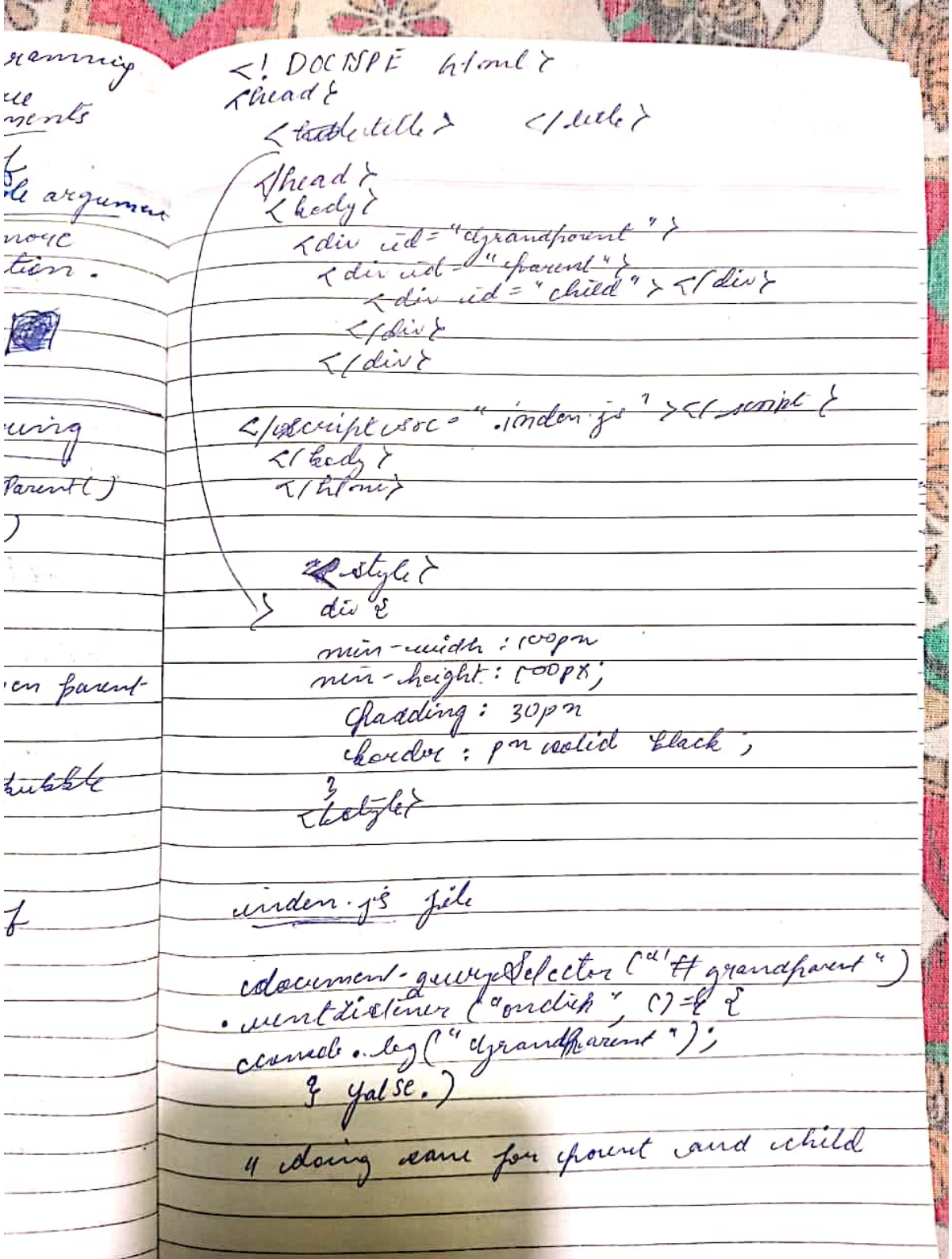
```
let multiply = function (x, y) {
    console.log (x * y);
}

let multiplyByTwo = multiply.bind (this, 2);
multiplyByTwo (5)
```

```
let multiplyByThree = multiply.bind (this, 3);
multiplyByThree (5);
```

Currying using bind

Currying using closure

```
function multiply (x) {
    return (y) {
        console.log (x*y);
    }
}

let multiplyByTwo = multiply (2);
multiplyByTwo (3);

let multiplyByThree = multiply (3);
multiplyByThree (2);
```



<!DOCTYPE html>
<head>
<title></title>
</head>
<body>

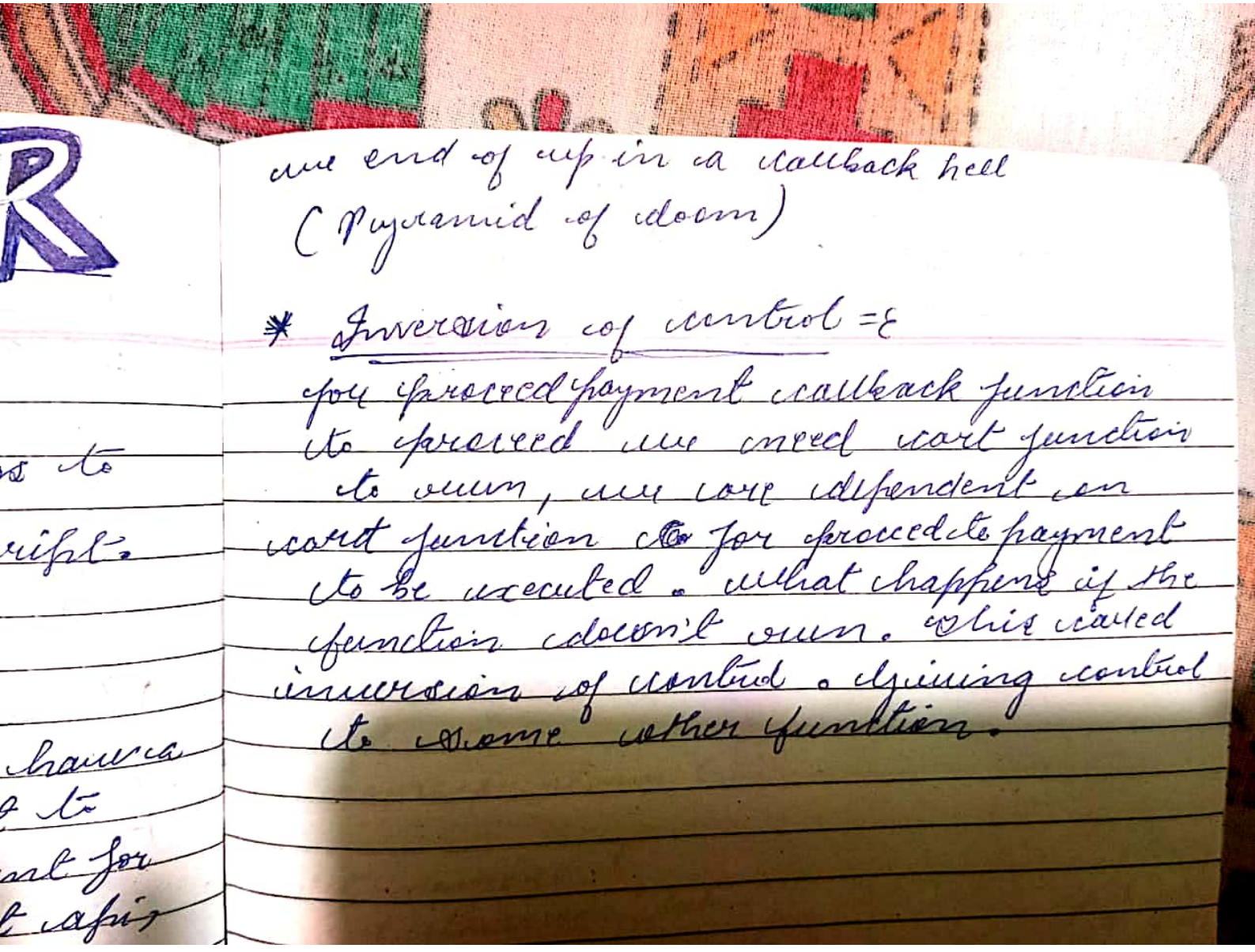
<script src="index.js"></script>
</body>
</html>

<style>
div {
min-width: 100px;
min-height: 100px;
padding: 30px;
border: 1px solid black;
}
</style>

index.js file

document.querySelector("#grandparent")
.addEventListener("click", () => {
console.log("grandparent");
}, false);

// doing same for parent and child



we end of up in a callback hell
(Pyramid of doom)

* Inversion of control =

for proceed payment callback function
to proceed we need card function
to run, we are dependent on
card function for proceed to payment
to be executed. what happens if the
function doesn't run. this called
inversion of control. giving control
to some other function.

Apply, Call, Bind

These are higher order function -

```
let name = {
    firstname : "Akash",
    lastname : "Kuna",
    printname() : function () {
        console.log (this.firstname + " " + this.second name);
    }
}
name.printname();

let name2 = {
    firstname : "Sachin",
    lastname : "Tendulkar";
}
// function borrowing
name.printname.call(name2); - here we are borrowing function from name.
```

Allows a method directly by passing argument, which points to this variable method inside that method.

```
let name = {
    firstname = "Ramesh",
    lastname = "Kuna";
}

let printname() {
    consol.log (this.firstname + this.second lastname);
}

printname.call(name) - { name is borrowing function printname
```

```
let name = {
    firstname : "Ramesh",
    lastname : "Kumar",
};

let printname = function (homeTown, state) {
    console.log("this.firstname + " " + this.lastname
    + from " + homeTown + "," + state + state);
};

printname.call(name, "Dehradhun", "Jharkhand");

printname.apply(name, ["Dehradhun", "Jharkhand"]);
```

passing argument as array list

Only difference in call and apply is that it difference in passing of argument.

// Bind

```
let printMyname = printname.bind
    (name, "mumbai", "Maharashtra");
console.log(printMyName);
printMyname();
```

printname

Ramesh from Mumbai state Jharkhand

```
funct (homeTown, state) {
    console.log( - - -
    - - - - )
}
```

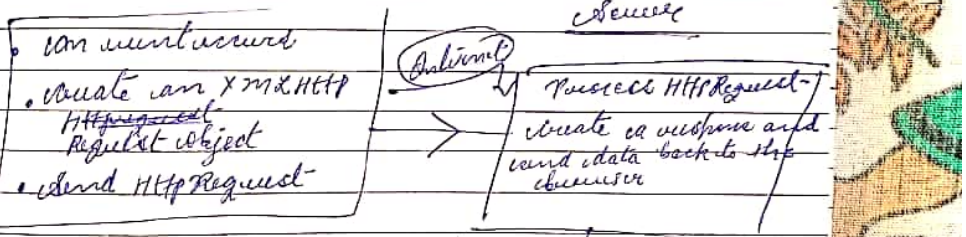
bind keep copy of a method and use it later rather than directly using it like call.

AJAX

Asynchronous Javascript and XML

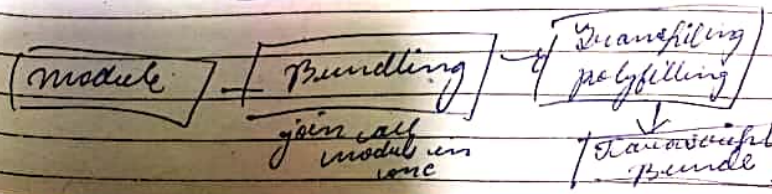
AJAX just uses of combination of a browser built in XMLHttpRequest object (to request data from a web server)
Javascript and HTML DOM (to display or use the data)

- Ajax allows web pages to update asynchronously by exchanging data with a web server behind the scene.
- Update part of a webpage without reloading the whole page.



Module — Contains open-source packages to include in your code (eg: React, jQuery, Leaflet etc)

npm node JS



```
const usersobj = users.map((user) => {
    return {firstname: user[0],
            lastname: user[1]
    }
});

console.log(usersobj);

// using destructuring

const usersobj = users.map(([firstname, lastname]) => {
    return {firstname: firstname, lastname: lastname}
});
```

Polyfills - A javascript code snippet or libraries, to replicate the missing functionality in older browsers.
& Written using standard javascript compatible with old browser.

ES6 - ECMAScript 6 (ES6)
A browser which does not support ~~polyfill~~ promise will use polyfill object.

Parcel - for bundling process.
(Babel)

Transpiling / polyfilling → Convert modern Javascript to ES5

Destructuring (ES6)

(no need of repeating)

Array destructuring

```
const arr = [2, 4, 5]
let [a, b] = arr;

console.log (a, b)
```

o/p: 2, 4 ~~+ b~~

```
let [a, b, c] = arr;
console.log (a, b, c));

const arr = [2, 4, 5, 6, 7, 8];
let [a, b, ...rest] = arr;
```

a = 2 b = 4 rest = 5, 6, 7, 8 (all other rest.)

```
let [a, b, , , ...rest] = arr;
```

a = 2 b = 4 5 6 ...rest = 7, 8

```
console.log [a, b, rest)
```

o/p: 2, 3 [7, 8, 8]

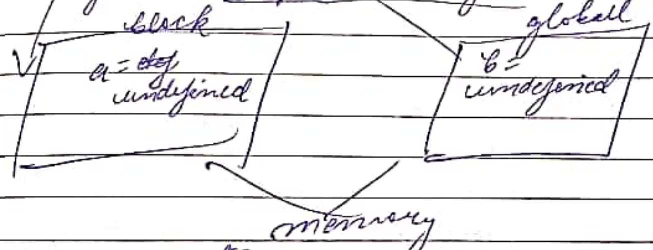
5, 6 will not be printed. (because we left it blank)

For array destructuring use [ ] - square bracket.

"let" vs "var"

console.log(b); – undefined
let a = 10; – block scope (cannot access before declaration)
var b = 100; – global scope (can be accessed without declaration)

if accessed before declaration, then it go in temporal deadzone



hoisting

from till it have given some value it is in temporal zone (can be access a variable in temporal deadzone)

window object – global object.

const a ; } cannot be done
a = 10 ; } (missing initializing in const declaration)

const a = 10; (correct)

*type error

* temporal deadzone – reference error
syntax error → syntax is wrong.

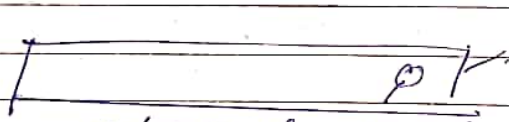
use 'const and let' in normal days.

return function (... args2) {
obj.apply(args1, [... params, ... args2]);
}
}

Debouncing & Throttling

- for optimizing performance of a webapp by limiting the rate of execution of function calls.

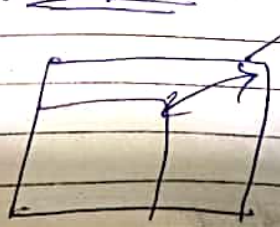
Example



no api call after each key stroke when there is a pause - it does api call. (Debouncing) \ diff btw two keystrokes > 300ms

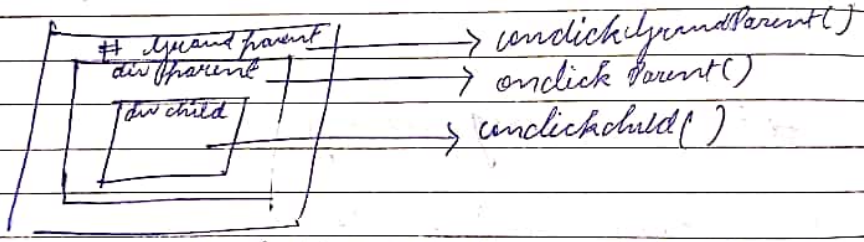
Throttling - it does api call after 300 ms.

Example -



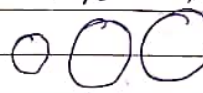
Currying in javascript programming technique in Javascript where function with multiple arguments is transformed into a series of function, each taking a single argument. This allows you to create more specialized and reusable function.

Event Bubbling & Capturing

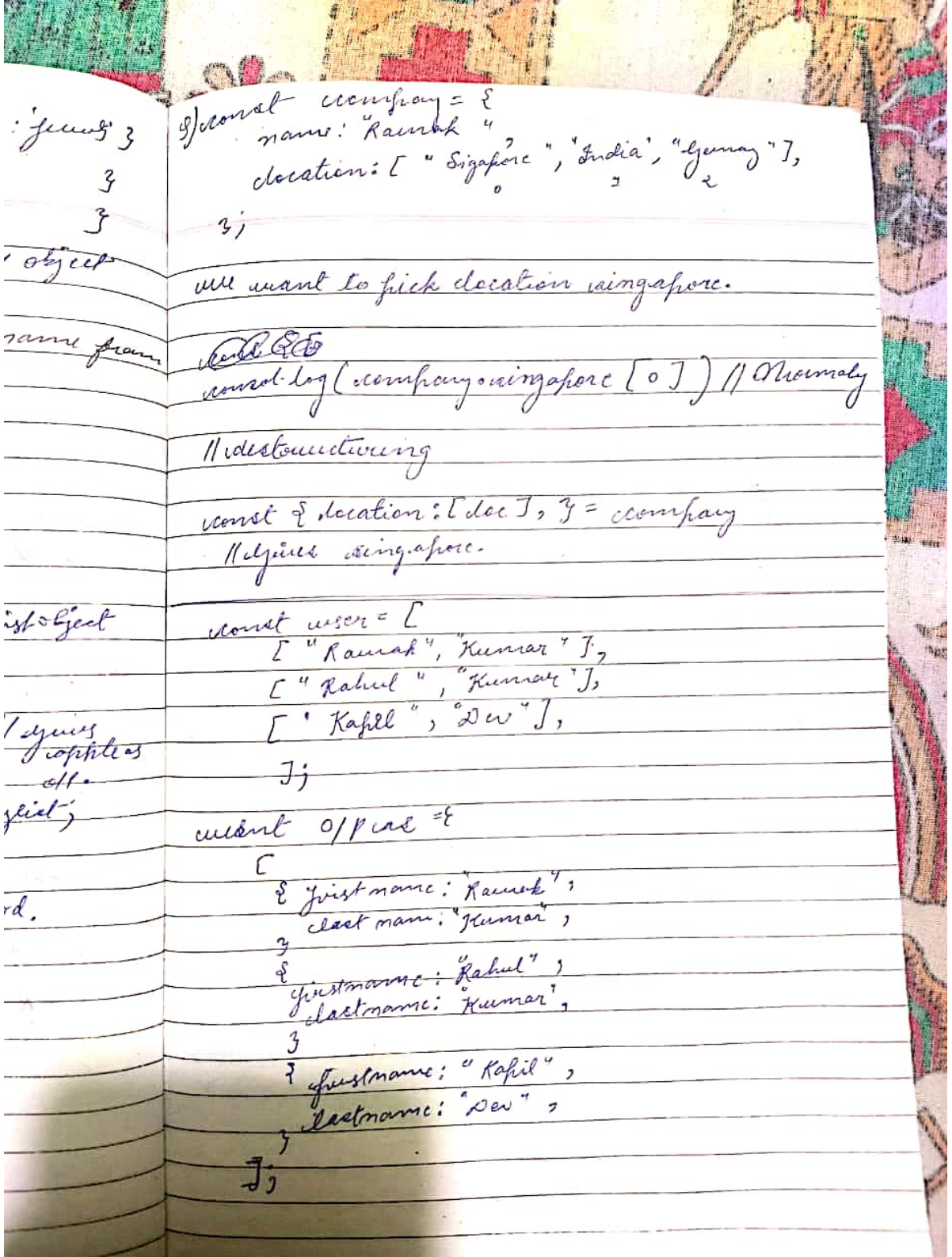


First child is called on click then parent then grandparent.

Small to up in DOM like a bubble rises



and capturing is opposite of bubbling.



```
//const company = {
    name: "Ramesh",
    location: [ "Singapore", "India", "Germany" ],
};

/// want to pick location singapore.

console.log(companysingapore[0]) // Anomaly

//destructuring

const { location: [loc] } = company
//gives singapore.

const user = [
    [ "Ramesh", "Kumar" ],
    [ "Rahul", "Kumar" ],
    [ "Kapil", "Dev" ],
];

want o/p as {
[
  { firstname: "Ramesh",
    last name: "Kumar",
  }
  {
    firstname: "Rahul",
    lastname: "Kumar",
  }
  { firstname: "Kapil",
    lastname: "Dev",
  }
];
```

Callback Hell

* callback are wonderful things to handle async operation in java script.

What is callback hell

Suppose in a ecommerce app we have a cart api, after adding element to cart, we need to make payment for which we need make payment api, Now after show order summary for which order api.

```
const cart = ["shoe", shirt, "pant"];

api.cart(cart, function () {
    api.proceedPayment();
});

api.proceedPayment(function () {
    api.showOrder()
});
```

callback function.

Here our code is growing horizontally instead of growing vertically

rest operator - ... getting data as a whole from any source.

spread operator

```
let arr = [3, 5, 8]
let obj = { ...arr };
console.log(obj);
```

o/p

```
{
  0: 3
  1: 5
  2: 8
}
```

it will convert array in key value pair like an object.

```
function sum(a, b, c) {
  return a+b+c;
}
console.log(sum(...arr));
```

passing array as a whole using spread operator.

Scanned with CamScanner

Set Timeout-

```
var fun = setTimeout(() => {
    console.log("Ramesh");
}, 5000);
```

— does not guarantee, may take more or less time / depends on call stack.

```
1 console.log("Start");
2 setTimeout(function abc() => {
3     console.log("callback");
4 }, 5000);
5 console.log("End");
  - - - - - - more code (takes) 10 sec to execute.
```

start
End
Callback. (after 10 sec) (because in call stack GEC was busy in execution the line after line 5 which took 10 sec and when call stack was empty then our abc() function got chance to execute in call stack. (Here we blocked the main thread)

console

GEC

callstack
global Execution content.

Web API Environment for SetTimeout

* setTimeOut()
* DOM APIs
* fetch()
* console

callback Queue

abc()

CQ will only when call stack is empty.

Promise

* ca container for future value.
* ca empty object
* can object represent can eventually completion of async operation
* states → pending
  → fulfilled
  → rejected

* promise constructor (creating object of promise)

```
let promise = new promise (function (
  resolve, reject ) => {

});
```

.then (
(success)

.catch (error) {
(rejected) console.log (error);
}

```
const groceryList [
{item: 'Apple', price: 2$, category: 'fruits'},
{ - - - - - },
{ - - - - - }
];  ——— we have a array of object

// we want to pick item name from first object

const ans = groceryList[0].item
console.log(ans)

// instead do this

console[apple] = groceryList // gives first object


const [{item:apple}] = groceryList // gives apples obj.

const [ , ,{item}, ...rest] = groceryList;

gives third item leaving first two

gives rest all object from 3rd.
```

object Destructuring

```
const employee = {
    id: 1,
    name: "Ramnak",
    age: 24,
    city: "Jamshedpur",
    dep: {
        pid: 12,
        salary: 5000,
    },
};  // closing of object
```

// destructuring to access name and age.

```
employee.name = name;
employee.age = age;
console.log(name, age)
```

repetition

to avoid this

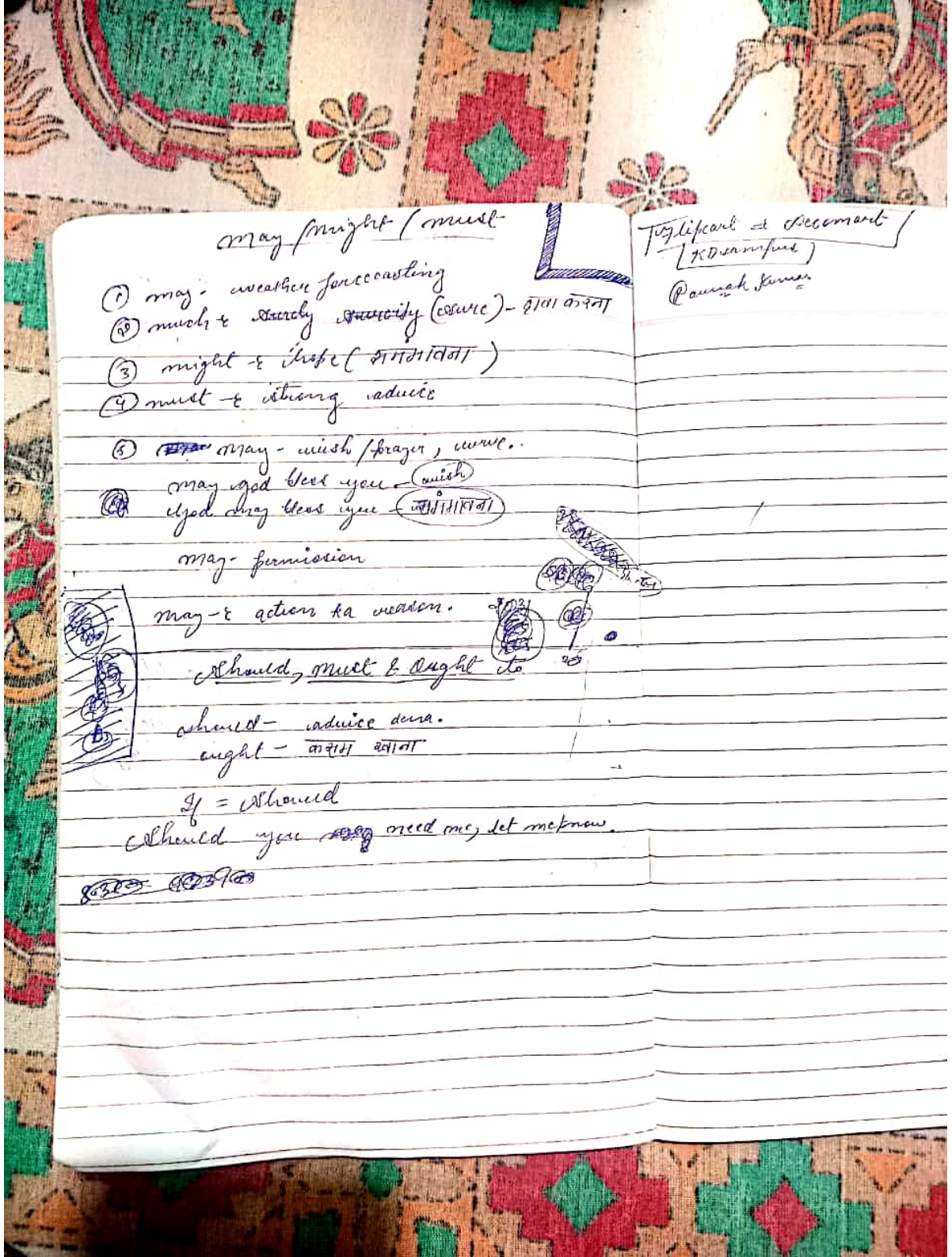
ES6 destructuring

```
const { name, age } = employee;
(console.log(name, age);
```

OR

```
const {name: fullname } = employee
```

here we are changing name to full name



may / might / must

(1) may - weather forecasting
(2) must & surely ~~surely~~ (sure) - पक्का करना
(3) might - & hope (शायद)
(4) must - & strong advice
(5) ~~may~~ may - wish / prayer, curse.
may god bless you - (wish)
god may bless you - (आशीर्वाद)
may - permission
may - & action ka reason.

should, must & ought to

should - advice dena.
ought - करना चाहिए

If = should
should you need me, let me know.

Flipkart & Decemart
(KDcampus)
@aunak kumar

Scanned with CamScanner

= 4 - 1 (3)
5 - 1   m - 1 - 2
4 - 3

0 1 2 3 4
| 1 2 3 4 5 | 0 1 2 3 4 5

which is valid comment

<!-- -->

Q) which of following is correct format to declare doctype.

Q) <! >

Q) <!DOCTYPE html>
<html>
<head> <title> </title>
<body> <p> — </p>
</body>
</html>

Q) 4 3) 4, (5), 6, 7

Topics
(1) CSS (2) Box Model (4) Color (6) Font
(3) Layout (5) Nav Form Table
Image.

Raunak

Raunak

(1) missing no. in a matrix
(1) find row with maximum no of 1's
(3) add two matrix
multiply
pre/post/in Order without recursion.
No of leaves in a binary tree.
find nth fibonacci numbers?

Q) Print no of trailing zeros in a factorial no N?

Q) Sum of all prime b/w 1 to 1000.

1	2	2
3	1	(2)
2	3	1

5 (5)
(5) 0

for i=0; i<n; i++   n.size()
j=0; j<m; j++   arr[i].size();

	0	1	2
0	1	2	0
1	3	4	2
2	2	3	1

notzeros 6 - 3 = (3)
zero

#include <set>

Block the (main thread) setTimeout.

```
console.log ("start")

setTimeout (function() => {
  console.log ("callback");
}, 5000);

let startDate = new Date().getTime();
let endDate = startDate();

while(endDate < startDate+10000) {
endDate = new Date().getTime();
}

console.log. ("while End");
```

start
while End
callback

O/P

Java Script is a synchronous single threaded language. It has only one call stack. So it is Interpreted language run inside browser does not have wait to compile.

electronic device.

Computer is an x which is used to store, retrieve and process data.

Data is raw materials.

Sequence of instruction is called a program.

Basic function of Computer

1. Input
2. Save — (save as cache, using memory address)
3. Execution
4. Output
5. Control.

Verb

IE

R R R

- Helping verb
  - Primary Helping
  - Modals [can, could, shall, should, will, would, must, ought to, may, might, need, dare, used to.]
- main verb

can, could → to tell your ability

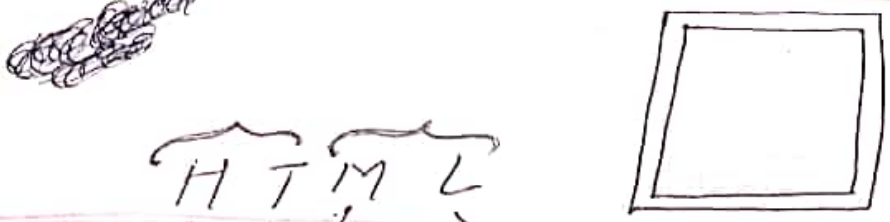
① I cannot come today / I can come today.
② Doctor could not treat him.

can → present ability, could → past ability.

with possible sentence → do not use used. could. (use might, can, manage to, able to)

doctors ~~were~~ were / doctor was.

→ If sentence start with V2 form further in sentence we will use V2 only.



HTML

Hypertext

Markup

Language

<!-- . . . -->

markup language + metadata + data that store info of different data.

HTML + Structure language.
Webpage file displayed in web browser

DOCTYPE
<! Doctype html>
<html>
<head>

<head>
<title> </title>
</head>
<body>

</body>
</html>

let s = "my name is Raunak";

console.log(s.~~last~~IndexOf("a"));

console.log(s.lastIndexOf("a"));

console.log(s.endWith("Raunak"));

output

true

console.log(s.include("is"));

output

True.

Substring function

let s = "my name is RAUNAK"

console.log(s.substring(0, 4));

output

my_n

let s = "my name is ROCKY";

console.log(s.split(" "));

output

splitting according to space.

1: "my"
2: "name"
3: "is"
4: "ROCKY"

console.log(s.replace("is", "was")); (replacing is with was)

substr() function

let s = "my name is RAUNAK";

console.log(s.substr(3, 4));

output: name (3 = start, 4 = length)

console.log(s.substr(0, 5));

output: my na

Substring and substr are two different functions.

Comparing

a = 15 b = 16

(a == b) — true (compare only value)

(a === b) — true (compare value and data type both)

let a = 15, b = "15";

(a === b) — false (is in int and 15 in string is diff.)

(a == b) — true

let n = 40; const a = 40
(can be changed) (cannot be changed)

let name = "RAUNAK"; let surname = "KUMAR";

console.log(`Hey, my name is ${name}`)
this is backticks curly bracket

console.log(`My full name is ${name} ${surname}.`)

const n = `My name ${name} ${surname}`

console.log(`My statement : ${n}`)

let n = `"RAUNAK"`;
console.log(n) → output: "RAUNAK"

Type Conversion

let a = 45;
(1) console.log(typeof a.toString(a));
(2) a = a.toString();
console.log(a);

let s = "Raunak";
console.log(typeof Number(s))

getName();

var getName = () => {
console.log("Rahul")
}

O/P = getName() is not a function
(if we store function in a variable)
(it stored memory for getName() as a variable with keyword "undefined").

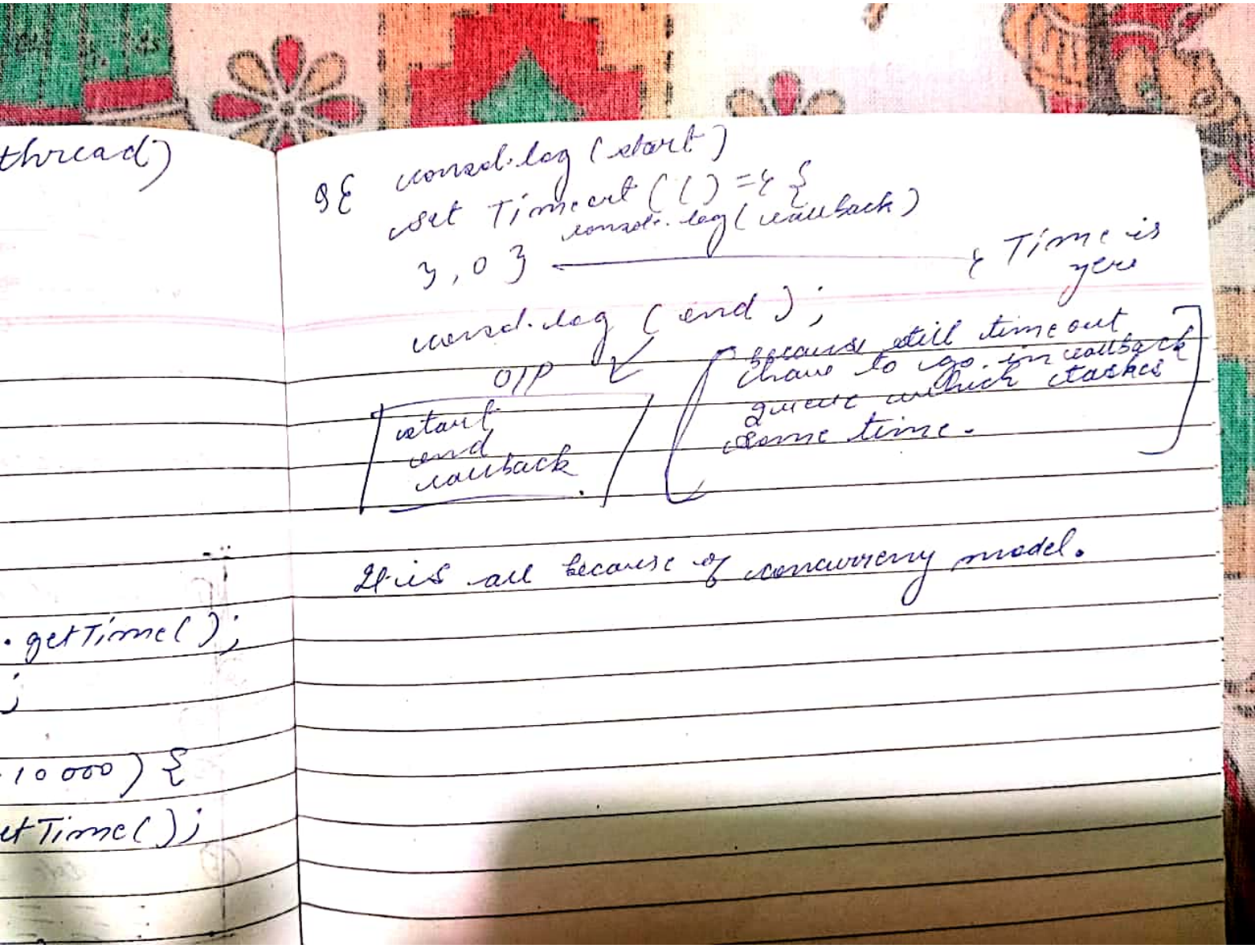
map()
→ transform an array or reduce it to new array based on some logic

```
const arr = [1, 2, 3];
const arr = [2, 4, 6] // array is doubled.

function double(n) {
return 2 * n;
}

const output = arr.map(double);
o/p = [2, 4 6].
```

myfilter function()

Basically filter the array based on some parameter.

```
const arr = [2, 3, 6, 8, 1, 0]

const output = arr.filter((n) => { n < 3 });
console.log(output);
```



thread()

. getTime();

; 10000) {

tTime());

console.log(start)

setTimeout(() => {

console.log(callback)

}, 0) ← Time is zero

console.log(end);

O/P

start
end
callback

because still timeout
have to go to callback
queue which tasks
same time.

It is all because of concurrency model.

Hoisting

```
getName();
console.log(n); // calling before their initialized.
console.log(getName)

var n = 7;
function getName() {
    console.log("Raunak");
}
```

initialization

o/p :-

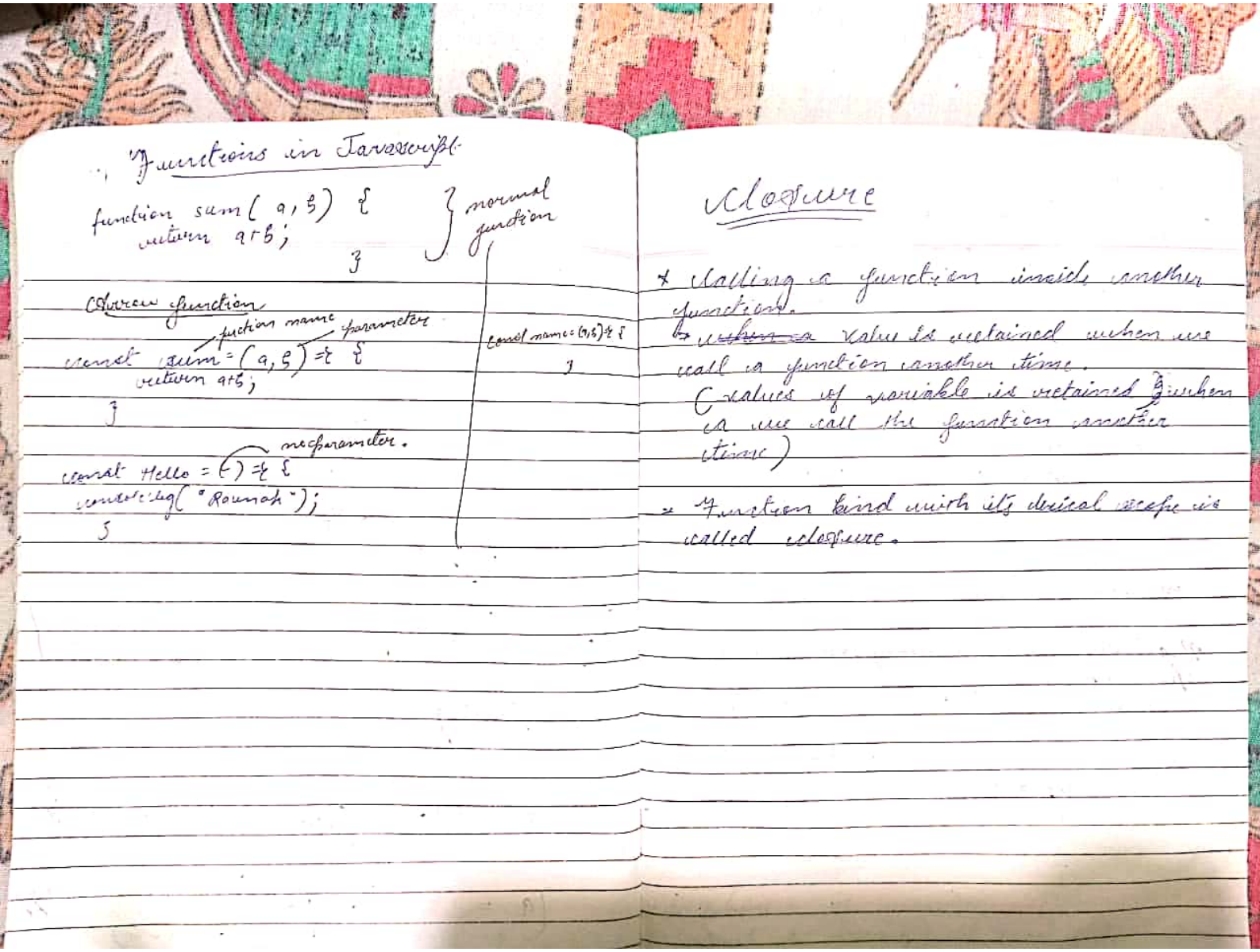
```
Raunak
undefined
ƒ getName() {
    console.log("Raunak");
}
```

```
console.log(n);
```

var n = 7 — if we do not write this.

o/p = n not defined.

* In phase 1 of memory creation javascript gives a memory to each variable and function and use a special keyword undefined for these variable.



Functions in Javascript

```
function sum(a,b) {
    return a+b;
}
```
} normal function

Arrow function

function name — sum — parameter

```
const sum = (a,b) => {
    return a+b;
}
```

no parameter.

```
const Hello = () => {
    console.log("Roumah");
}
```

```
const name = (a,b) => {
}
```

Closure

* Calling a function inside another function.
* When a value is retained when we call a function another time.
  (values of variable is retained when we call the function another time)
* Function bind with its lexical scope is called closure.

I/P = { 'a', 'b', 'c', 'a', 'b', 'c', 'd' }

O/P = { a: 2, b: 2, c: 2, d: 1 }

```
function value ( array ) {
const out = { };// empty object
array.map(curr => {
out[curr] = (out[curr] || 0) + 1;
})
return out;
}
console.log (value (data));
```